

Backend Production Readiness Audit

Date: 2026-09-30 Project: AI-first social-commerce / F-commerce backend Scope checked: docs/, tasks/task-1.txt through tasks/task-16.txt, pdfs/ index, backend routes, migrations, API docs, tests, Docker/deploy files.

Executive Rating

Backend production readiness: 58 / 100

Feature implementation coverage: 68 / 100 Production operations readiness: 49 / 100 Security/compliance readiness: 55 / 100 Performance readiness for 1000 users: 52 / 100

Meaning: the backend is a strong prototype / early beta backend. It has real module boundaries, tenant-aware tables, auth, roles, audit events, order/payment/inventory flows, webhooks, jobs, and API documentation. It is not yet a top-class production backend because real provider integrations, DB-backed CI, observability, legal surfaces, backups, hard SLOs, and load-tested performance are not complete.

Recommended launch status: private pilot only, not public SaaS production.

Evidence From This Repository

Found:

- 149 backend route handlers.
- 35 migrations.
- 19 test files.
- Route families for auth, shops, assets, catalog, inventory, onboarding, storefront, orders, delivery, payments, customers, marketing, Meta, webhooks, analytics, AI, jobs, health, and system.
- API docs check for 21 route files.
- Docker Compose with API, worker, Postgres, Redis.
- Fastify security basics: Helmet, CORS, multipart limits, rate limit.
- Postgres migrations for tenant data, audit events, order timelines, payments, COD settlement, Meta, comments, analytics, AI commands, jobs, auth challenges, publish config, integrations, idempotency, and audit immutability.

Verification run:

```
cd backend
npm run check
```

Result:

```
API docs check passed for 21 route file(s).
TypeScript build passed.
Onboarding integration checks passed.
DB checks skipped because database is not reachable.
```

Important caveat: the default check did not prove the database smoke tests in this run. Production readiness cannot be higher until the DB test suite runs in CI against Postgres.

What Has Been Built

Foundation

Built:

- Modular monolith structure.
- API version prefix /api/v1.

- Request context hook.
- PostgreSQL connection pool.
- Docker local stack with API, worker, Postgres, Redis.
- Health/readiness endpoints.
- Module registry/system module endpoint.
- API documentation rule and checker.
- Migrations as explicit deploy step.

Good:

- This matches the right early architecture. For this product size, microservices would add cost before value.
- Module boundaries are clear enough to split later.

Missing:

- Redis is configured but not deeply used for sessions, queues, locks, rate limits, or cache.
- No production-grade distributed tracing, metrics endpoint, or error tracker.
- No staging/prod environment contract beyond env vars.

Auth, Shops, Roles, Settings

Built:

- User registration and login.
- Scrypt password hashing.
- Bearer-token sessions stored hashed in DB.
- Session revocation.
- Email/phone verification challenge model.
- Password reset model.
- Fixed roles: owner, admin, sales, packer, marketer, accountant.
- Permission checks per shop.
- Team management.
- Owner transfer.
- Shop settings.
- Audit viewer.
- Billing read model placeholders.

Good:

- Passwords and tokens are not stored raw.
- Role model covers small seller teams.
- Permission guard is centralized enough to be useful.

Missing:

- Real OTP/email/SMS delivery provider.
- MFA for owners/admins.
- Account lockout policy beyond rate limits.
- Password policy and breached-password screening.
- Cookie-based secure browser sessions if admin frontend uses browser auth.

- CSRF strategy if cookies are introduced.
- Device/session risk scoring.
- Admin security page with recent logins, revoke all sessions, and integration access review.

Catalog, Inventory, Assets

Built:

- Products and variants.
- Product images through asset metadata.
- Local file storage.
- Product import/export route shape.
- Inventory ledger.
- Stock adjustment.
- Stock reservation/release in order flows.
- SKU conflict handling.
- Product snapshots in orders.

Good:

- Ledger-first inventory is correct.
- Product snapshots protect historical orders.
- File metadata is shop-owned.

Missing:

- S3/R2 object storage implementation.
- Malware/image scanning.
- Image resizing/CDN pipeline.
- Native CSV/XLSX upload parsing and mapping.
- Multi-warehouse/bin support. Not needed for V1 unless seller demand appears.

Storefront, Checkout, Orders

Built:

- Public storefront shop and product reads.
- Checkout submit.
- Order tracking lookup.
- Order drafts.
- Draft extraction placeholder.
- Draft confirm.
- Order list/detail.
- Status pipeline.
- Order timeline.
- Bulk preview/export/book route shape.
- Cancellation/return stock behavior.
- Checkout idempotency.

Good:

- Critical stock checks are server-side.
- Timeline and audit are present on important status changes.
- Public tracking is not a plain open order detail endpoint.

Missing:

- Real checkout-link expiry enforcement and public flow completeness.
- Full issue queue UX/data model for all order problems.
- Invoice/packing slip generation.
- Return/refund operational workflow end to end.
- Fraud/duplicate/risk model beyond simple checks.

Inbox, Comments, AI

Built:

- Conversations and messages.
- Assignment.
- AI draft records with review states.
- Comment posts/rules/leads/moderation.
- Meta webhook ingestion with signature verification.
- AI command records, risk classification, approvals.
- AI ad creative records, brand rules, safety warnings.

Good:

- AI is mostly suggest/review mode, not trusted truth.
- Risky actions route toward approval.
- Source/reference fields exist in the design.

Missing:

- Real LLM provider integration.
- Real Messenger/Instagram outbound send.
- Real AI order extraction.
- One-action gateway for all AI execution.
- Undo/rollback executor map.
- Prompt/version logging and eval set.
- Abuse/safety policy operations.

Delivery, Payments, COD

Built:

- Courier accounts.
- Manual shipment booking.
- API shipment queue placeholder.
- Tracking events.
- Failed delivery and reschedule.
- Webhook-preferred status guard.
- Manual/COD payments.

- Proof asset reference.
- Refund records.
- COD settlement import/matching.
- Payment idempotency.

Good:

- Manual mode is a real V1 fallback.
- Payment events preserve history.
- Marked-paid does not pretend bank confirmation.

Missing:

- Real courier provider adapters.
- Real payment gateway/bank confirmation.
- Shipping label/booking sheet export.
- Settlement dispute workflow.
- Finance-grade reconciliation reports.

CRM, Marketing, Analytics, Meta

Built:

- Customers, addresses, tags, consent.
- Merge preview/apply.
- Coupons.
- Segments.
- Broadcast drafts/previews/approval.
- Retention report.
- Analytics events and reports.
- Meta OAuth shape and encrypted token storage path.
- Meta catalog sync preview.
- Campaign stats import.

Good:

- Consent exists.
- Broadcast approval exists.
- Meta token encryption design exists.

Missing:

- Actual campaign sender.
- Consent proof/audit robust enough for legal disputes.
- Real Meta CAPI event sender.
- Real catalog sync sender.
- Attribution reconciliation beyond imported stats.
- Warehouse-quality analytics definitions and rollups.

Comparison To Strong Production Commerce Systems

Medusa documents a layered commerce architecture: HTTP routes, workflows, domain modules, and PostgreSQL data store. It also separates server and worker instances for production, with PostgreSQL and Redis as core services. Source: <https://docs.medusajs.com/learn/introduction/architecture> and <https://docs.medusajs.com/learn/deployment>

commercetools presents commerce as API-first business capabilities: products, carts, orders, customers, pricing, promotions, and integrations. Source: <https://docs.commercetools.com/guides/implementation-guide/architecture>

Shopify documents idempotency for retry-safe commerce actions and API rate limits for platform stability. Sources: <https://shopify.dev/docs/api/usage/idempotent-requests> and <https://shopify.dev/docs/api/usage/limits>

This project already follows the correct shape in several ways:

- API-first backend.
- Separate public/storefront and dashboard route families.
- Modular monolith with clear commerce domains.
- Postgres truth.
- Worker process.
- Audit/timeline on important operations.
- Idempotency on checkout/payments.
- Rate limits and webhook dedupe.

Where top-class production systems go further:

- Workflows are explicit and reusable, not scattered route/service calls.
- Every external integration has adapter contracts, timeouts, retries, dead-letter, replay tools, and dashboards.
- Observability is a product feature for developers and operators.
- Admin dashboards are built for repeated operations: queues, filters, bulk safe actions, saved views, exception handling, audit, and undo where possible.
- CI proves migrations and smoke tests against real Postgres/Redis.
- Privacy/legal surfaces are first-class, not footer text added later.
- Performance is measured with load tests and budgets per endpoint.

Production Gaps Blocking Public Launch

Must Fix Before Real Production

- Run DB-backed CI.
- Start Postgres/Redis in CI. - Run all smoke tests, not only onboarding. - Fail on migration drift.
- Add real observability.
- JSON logs already exist through Fastify/Pino; standardize fields. - Add request latency histograms, route error counters, DB query timing, worker job metrics. - Add alerting: API 5xx, p95 latency, queue backlog, DB connection pressure, webhook failure, payment/audit write failure.
- Harden auth.
- Real OTP/email/SMS delivery. - MFA for owner/admin. - Login throttling by identifier and IP. - Password reset abuse limits. - Secure cookie/session option if browser admin uses cookies.
- Finish storage.
- S3/R2 driver. - Signed uploads or server-side upload path. - MIME sniffing, image validation, malware scanning for proof files. - CDN URLs.
- Finish provider adapters.

- Meta outbound messages/comments. - Meta CAPI. - Meta catalog sync. - Courier booking/tracking adapters. - Email/SMS/notification provider. - Optional payment gateway later.
- Add production migration/backup discipline.
- Backup schedule. - Restore drill. - Migration rollback plan. - Zero-downtime migration rules for additive changes.
- Add legal/compliance pages and data controls.
- Privacy policy. - Cookie policy and consent. - Terms and conditions. - Refund/return policy. - Shipping/delivery policy. - Cancellation policy. - Data deletion/export requests. - Data retention settings.
- Load test.
- Prove 1000 concurrent active users or expected RPS. - Track p50/p95/p99 latency. - Test checkout/order confirm under write contention. - Test webhooks burst ingestion.
- Add admin operational screens.
- Job queue/dead-letter viewer. - Failed webhook replay. - Integration health. - Audit search. - Risky action approval inbox. - Billing/usage controls.
- Unify risky action gateway.
- Cancel, refund, mark paid, bulk message, courier booking, integration disconnect, owner transfer, export, AI command execution. - Same path: permission, preview, approval, idempotency, execute, audit, notification.

Legal, Policy, And "Tricky" Things To Add

These were not fully specified in the PDFs/tasks but are needed for a real-life ecommerce SaaS.

Public Store Legal Pages

Add seller-configurable pages:

- Privacy Policy.
- Cookie Policy.
- Terms and Conditions.
- Return and Refund Policy.
- Shipping and Delivery Policy.
- Cancellation Policy.
- Contact/About Seller.
- Payment/COD terms.
- Warranty policy if seller offers warranty.
- Age-restricted product policy if relevant.

FTC consumer guidance says buyers should be able to find privacy policy, user agreement/terms, shipping fees, and return/refund rules. Source: <https://consumer.ftc.gov/articles/online-shopping>

FTC prompt delivery guidance says sellers must ship within promised time, or within 30 days if no promise is made, and must notify buyers of delays with cancellation/refund options. Source: <https://www.ftc.gov/business-guidance/resources/business-guide-ftcs-mail-internet-or-telephone-order-merchandise-rule>

Cookie And Tracking Consent

Add:

- Cookie banner.
- Consent categories: necessary, analytics, marketing.
- Meta pixel/CAPI consent gate.

- Consent event log.
- "Do Not Sell or Share" / opt-out link where applicable.
- Global privacy control handling where applicable.

CCPA/CPRA rules effective 2026 include opt-out mechanisms for online personal information sale/sharing. Source: https://cppa.ca.gov/regulations/pdf/ccpa_statute_eff_20260101.pdf

Data Privacy Operations

Add:

- Customer data export.
- Customer deletion/anonymization workflow.
- Retention policy per shop.
- Staff access log.
- Integration data-sharing disclosure.
- Vendor list.
- DPA-ready docs if selling SaaS to businesses.
- Subprocessor page.
- Privacy contact email.

Admin Safety

Add:

- Confirmation modal for risky actions.
- Reason required for cancel/refund/return/mark paid/owner transfer/disconnect.
- Maker-checker approval for high-risk operations.
- Export watermark/audit.
- Rate limit bulk messaging.
- Broadcast quiet hours.
- Consent proof before campaign send.
- Staff role templates with least privilege.

Finance And Tax

Add:

- Tax/VAT fields by country.
- Invoice numbering.
- Refund accounting.
- COD settlement variance report.
- Payment proof review queue.
- Currency rounding rules.
- Manual adjustment audit.

Trust And Anti-Abuse

Add:

- Fraud/risk flags.

- Duplicate order detection.
- Buyer blocklist with audit.
- Staff impersonation restrictions.
- Admin IP/device anomaly alerts.
- Webhook replay protection dashboard.
- File abuse scanning.

Permissions Needed

Current fixed roles are a good start. Add these permission groups:

- billing:read, billing:write
- integrations:read, integrations:write, integrations:disconnect
- ai:read, ai:write, ai:approve, ai:execute
- webhooks:read, webhooks:replay
- jobs:read, jobs:retry
- audit:read
- privacy:read, privacy:write
- legal:write
- reports:read, exports:run
- orders:cancel
- payments:refund
- payments:mark_paid
- delivery:book
- marketing:broadcast_approve

Recommended V1 role changes:

- Owner: all permissions, MFA required.
- Admin: all except owner transfer and billing payment method.
- Sales: inbox, customers, order draft/create, no refunds/export.
- Packer: order packing/shipping, inventory read, no payment access.
- Marketer: catalog read, marketing, analytics, no customer export unless approved.
- Accountant: payments, settlements, exports, no inbox message send.

200 ms Response Plan For 1000 Users

Target must be precise:

- Public storefront product/list reads: p95 under 200 ms from edge/CDN/cache.
- Dashboard list reads: p95 under 200-400 ms depending filters.
- Checkout/order confirm: correctness over speed; p95 under 500 ms is more realistic.
- AI/courier/payment external calls: async, never block buyer/admin request.

Minimum architecture:

- CDN in front of storefront assets/images.

- Cache public storefront shop/product pages for 30-120 seconds with stale-while-revalidate.
- Redis cache for hot storefront product lists and settings.
- Postgres indexes for every common filter: shop_id, status, created_at, order status, customer phone, SKU, conversation status, shipment status.
- Cursor pagination only; no deep offset pagination.
- Avoid N+1 queries in list endpoints.
- Use read models/materialized summaries for dashboard metrics.
- Queue all external calls.
- Keep webhook HTTP response fast: verify, persist, enqueue, return.
- Pool sizing: start API pool at 10 per instance; calculate total connections against Postgres max.
- Use PgBouncer if multiple API/worker instances.
- Separate API and worker processes.
- Add load balancer and at least 2 API instances.
- Add worker concurrency controls per queue.
- Add object storage/CDN for files.

Performance test plan:

- Seed 1000 shops, 100k products, 100k orders, 500k messages.
- Run k6/Artillery scenarios:
- storefront browse - product detail - checkout submit - dashboard order list - inbox list/detail - webhook burst
- Budgets:
- public GET p95 <= 200 ms - dashboard GET p95 <= 400 ms - checkout p95 <= 500 ms - API error rate < 0.5% - webhook ack p95 <= 100 ms
- Add query logging for slow queries over 100 ms.
- Fix indexes based on EXPLAIN ANALYZE, not guesswork.

Roadmap To 100 / 100

Exact 100 / 100 Backend Checklist

Use this as the build checklist. Rating reaches 100 only when these are implemented, tested, deployed, and documented.

1. Production Infrastructure

Do:

- Create separate local, staging, and production environments.
- Run API and worker as separate processes.
- Put API behind Nginx/load balancer with HTTPS.
- Use managed or hardened PostgreSQL.
- Use Redis for rate limits, idempotency, cache, and worker locks.
- Use S3/R2 object storage for images, proofs, invoices, and exports.
- Put CDN in front of storefront assets and product images.
- Add PgBouncer before scaling API instances.

- Add daily automated database backups.
- Run monthly restore drills.

Needed:

- Production .env schema.
- Secret manager or encrypted deploy secrets.
- Backup retention policy.
- Restore runbook.
- Deploy rollback runbook.

100/100 proof:

- A staging deploy can be destroyed and rebuilt from backup.
- A production deploy can roll back without data loss.

2. CI/CD And Test Proof

Do:

- Start Postgres and Redis in CI.
- Run all backend smoke tests against real DB.
- Run migrations from empty database.
- Run migration compatibility check against seeded old database.
- Run npm run check:api-docs.
- Run TypeScript build.
- Add seed data for large shops.
- Add load tests to CI nightly, not every commit.

Needed:

- GitHub Actions or equivalent pipeline.
- Test database URL.
- Seed script with 1000 shops, 100k products, 100k orders, 500k messages.

100/100 proof:

- CI fails if any DB smoke test fails.
- CI fails if a route lacks API docs.
- Nightly load test report is stored.

3. Security Hardening

Do:

- Add MFA for owner/admin.
- Add real email/SMS OTP delivery.
- Add login throttling by IP and identifier.
- Add password policy.
- Add password reset abuse protection.
- Add staff session/device management.
- Add secure cookie session option if frontend uses browser cookies.
- Add CSRF protection if cookie auth is used.

- Encrypt all integration tokens.
- Add token rotation for Meta/courier/payment providers.
- Add audit search for security events.
- Add file upload malware scanning.
- Add MIME sniffing and image decoding validation.
- Add export audit/watermark.

Needed:

- security:read, security:write, audit:read, integrations:disconnect permissions.
- Owner/admin MFA migration.
- Security event dashboard.

100/100 proof:

- Owner account cannot operate without MFA.
- Export/payment/refund/disconnect actions are permission checked, audited, and reasoned.
- Uploaded files cannot bypass MIME/size rules.

4. Legal, Privacy, And Compliance

Do:

- Add public legal pages for each shop:
- Privacy Policy. - Cookie Policy. - Terms and Conditions. - Return and Refund Policy. - Shipping and Delivery Policy.
- Cancellation Policy. - Contact Seller page.
- Add cookie banner and consent categories.
- Gate analytics/marketing pixels behind consent.
- Add buyer data export.
- Add buyer data deletion/anonymization.
- Add data retention settings.
- Add consent proof records.
- Add opt-out for campaigns.
- Add vendor/subprocessor disclosure page for SaaS admin.

Needed:

- legal_pages table.
- cookie_consent table.
- privacy_requests table.
- Admin UI for legal page editor.
- Public footer links and checkout links.

100/100 proof:

- Buyer can view legal pages before checkout.
- Marketing events do not fire before consent where required.
- Staff can export/delete buyer data through audited workflow.

5. Provider Integrations

Do:

- Add Meta outbound Messenger/Instagram send.
- Add Meta comment reply/DM sender.
- Add Meta CAPI sender.
- Add Meta catalog sync sender.
- Add first real courier provider adapter.
- Add courier webhook signature verification.
- Add email/SMS provider.
- Add real AI provider for:
- inbox draft, - order extraction, - command center, - ad creative draft.
- Add provider timeout, retry, backoff, dead-letter, and replay for each integration.

Needed:

- Adapter interface per provider type.
- Integration health table.
- Dead-letter/retry dashboard.
- Provider credential validation.

100/100 proof:

- If Meta/courier provider is down, order/chat data is not lost.
- Failed webhook can be replayed.
- Provider timeout does not block checkout/admin request.

6. Risky Action Gateway

Do:

- Route all risky actions through one gateway:
- cancel order, - mark paid, - refund, - bulk export, - bulk message, - courier booking, - integration disconnect, - owner transfer, - AI command execution.
- Gateway flow:
- permission check, - risk classify, - preview, - require reason, - optional approval, - idempotency key, - execute, - audit, - timeline/event emit.

Needed:

- action_approvals table.
- action_execution_logs table.
- Permission map.
- Preview builders.

100/100 proof:

- No high-risk mutation exists outside gateway.
- Every risky action has actor, reason, before/after data, and idempotency.

7. Observability And Operations

Do:

- Add structured request logs with request id, shop id, user id, route, latency, status.
- Add metrics:

- API latency/error rate, - DB query latency, - DB pool usage, - worker queue depth, - job success/failure/retry, - webhook accepted/failed/replayed, - provider latency/error, - audit write failures.

- Add alerts:

- API 5xx spike, - p95 latency high, - queue backlog, - worker down, - webhook failure spike, - DB connections high, - disk/storage near full, - backup failed.

- Add admin ops screens:

- job queue, - dead-letter, - webhook events, - provider health, - audit search, - failed payments/COD, - failed deliveries.

Needed:

- Prometheus/OpenTelemetry/Sentry or equivalent.

- Alert destination.

- Runbooks for top incidents.

100/100 proof:

- On-call can see why checkout is slow within 5 minutes.

- Failed jobs can be retried safely from admin.

8. Performance For 1000+ Users

Do:

- Add Redis cache for public storefront reads.

- Add CDN cache for product images and theme assets.

- Add stale-while-revalidate for storefront pages.

- Add cursor pagination everywhere.

- Add composite indexes for every dashboard filter.

- Remove N+1 queries from list endpoints.

- Add materialized/read models for analytics cards.

- Keep checkout/order confirmation strongly consistent.

- Keep external calls async.

- Add slow query logging.

- Tune DB pool and add PgBouncer.

- Run 2+ API instances and 1+ worker instance.

Needed:

- k6/Artillery tests.

- Seed data.

- EXPLAIN ANALYZE review for hot queries.

- Cache invalidation rules.

100/100 proof:

- Public storefront GET p95 <= 200 ms.

- Webhook ACK p95 <= 100 ms.

- Dashboard GET p95 <= 400 ms.

- Checkout p95 <= 500 ms.

- Error rate < 0.5% during load test.

9. Admin Product Quality

Do:

- Build real admin workflows:
- order queue with saved filters, - inbox assignment and review, - failed delivery queue, - payment proof review, - COD settlement issue queue, - broadcast approval, - AI approval center, - integration health, - audit search, - team/security settings.
- Add bulk action previews with per-item errors.
- Add undo only where rollback is real.
- Add empty/error/loading states.

Needed:

- Backend endpoints for saved views.
- Issue queue tables.
- Per-item bulk result format.

100/100 proof:

- Staff can run daily operations without touching database or logs.

10. Data Quality And Lifecycle

Do:

- Add retention policies.
- Add soft-delete/anonymization rules.
- Add immutable audit.
- Add duplicate customer/order detection.
- Add data export snapshots.
- Add data import validation with row-level errors.
- Add native CSV/XLSX import.
- Add analytics definitions table/document.

Needed:

- Import parser.
- Retention scheduler.
- Data deletion/anonymization jobs.

100/100 proof:

- Seller can import products with row errors.
- Buyer deletion request does not break historical financial/order records.

11. Release Discipline

Do:

- Add feature flags.
- Add migration review checklist.
- Add changelog.
- Add versioned API docs.
- Add staging signoff checklist.

- Add incident process.
- Add rollback process.
- Add dependency/security scanning.

Needed:

- Feature flag table or provider.
- Release template.
- Incident template.

100/100 proof:

- Risky feature can be enabled for one shop first.
- Bad deploy can be rolled back with known steps.

Scoring Ladder

Current: 58/100

- 65/100: DB-backed CI runs all smoke tests; S3/R2 storage added; backups configured.
- 70/100: metrics/alerts/logging added; staging/prod separation complete; restore drill done.
- 75/100: legal pages, cookie consent, privacy request workflow added.
- 80/100: Meta outbound, email/SMS, first courier adapter, real OTP delivery added.
- 85/100: AI provider integration with prompt logs/evals; risky action gateway complete.
- 90/100: ops dashboards for jobs/webhooks/integrations/audit; load tests passing.
- 95/100: CDN/Redis cache/PgBouncer/query tuning; 1000-user performance target proven.
- 100/100: pen test fixes complete, incident/backup/release drills complete, production runbooks complete, all critical workflows proven under load.

Best Order To Work

- Make CI run Postgres/Redis and all tests.
- Add S3/R2 storage and backup/restore.
- Add observability and alerts.
- Add legal/privacy/cookie system.
- Add real email/SMS OTP.
- Add Meta outbound and webhook replay.
- Add courier adapter.
- Add risky action gateway.
- Add load tests and performance indexes/cache.
- Add admin ops dashboards.

Do not start with microservices, app marketplace, complex BI, loyalty, multi-warehouse, or enterprise SSO. Those do not move production readiness as much as tests, security, legal, observability, providers, and load proof.

Phase A: Production Proof

Goal: make the current backend trustworthy.

- CI with Postgres/Redis.
- Run all DB smoke tests.

- Add SLO metrics and alerts.
- Add S3/R2 storage.
- Add backup/restore docs and drills.
- Add load test suite.
- Add seed data generator.
- Add security headers/CORS per environment.
- Add secrets rotation checklist.

Expected score after Phase A: 70/100.

Phase B: Provider Reality

Goal: remove placeholder integrations.

- Meta outbound send.
- Meta CAPI event sender.
- Meta catalog sync sender.
- Courier adapter interface and first real provider.
- Email/SMS provider.
- Real OTP delivery.
- AI provider with prompt logs, evaluation set, and safety rules.

Expected score after Phase B: 80/100.

Phase C: Legal And Admin Operations

Goal: operate like a serious SaaS.

- Legal page builder.
- Cookie consent and tracking controls.
- Privacy request workflows.
- Admin audit search.
- Job/dead-letter dashboard.
- Webhook replay dashboard.
- Integration health dashboard.
- Risky action approval center.

Expected score after Phase C: 88/100.

Phase D: Scale And Reliability

Goal: prove 1000+ users with 200 ms public reads.

- CDN/cache layer.
- Redis cache usage where safe.
- Query tuning.
- PgBouncer.
- Horizontal API instances.
- Worker queue concurrency and dead-letter policies.
- Staging environment with production-like data.

- Disaster recovery runbook.

Expected score after Phase D: 94/100.

Phase E: Top-Class Polish

Goal: mature product team quality.

- Feature flags.
- Release notes/changelog.
- Admin UX for all operational queues.
- Data warehouse export.
- Fine-grained permissions.
- Compliance documentation.
- Pen test.
- SOC2-style control evidence if selling to larger businesses.
- Uptime page and incident process.

Expected score after Phase E: 100/100 candidate.

Final Verdict

This backend is much more than a toy. It has the right architecture and many real commerce primitives. The best parts are multi-tenant data design, audit/timeline direction, inventory/order/payment care, API docs discipline, and conservative AI behavior.

The production gap is not "add more random features." The gap is proof and operations:

- real providers,
- real DB tests in CI,
- real observability,
- real legal/privacy surfaces,
- real storage/CDN,
- real load tests,
- real admin operations for failures.

Rating stays 58/100 until those are done. With the current foundation, getting to 80/100 is realistic without rewriting the backend.